

Data Structures

Unit 1: Introduction to data Structures

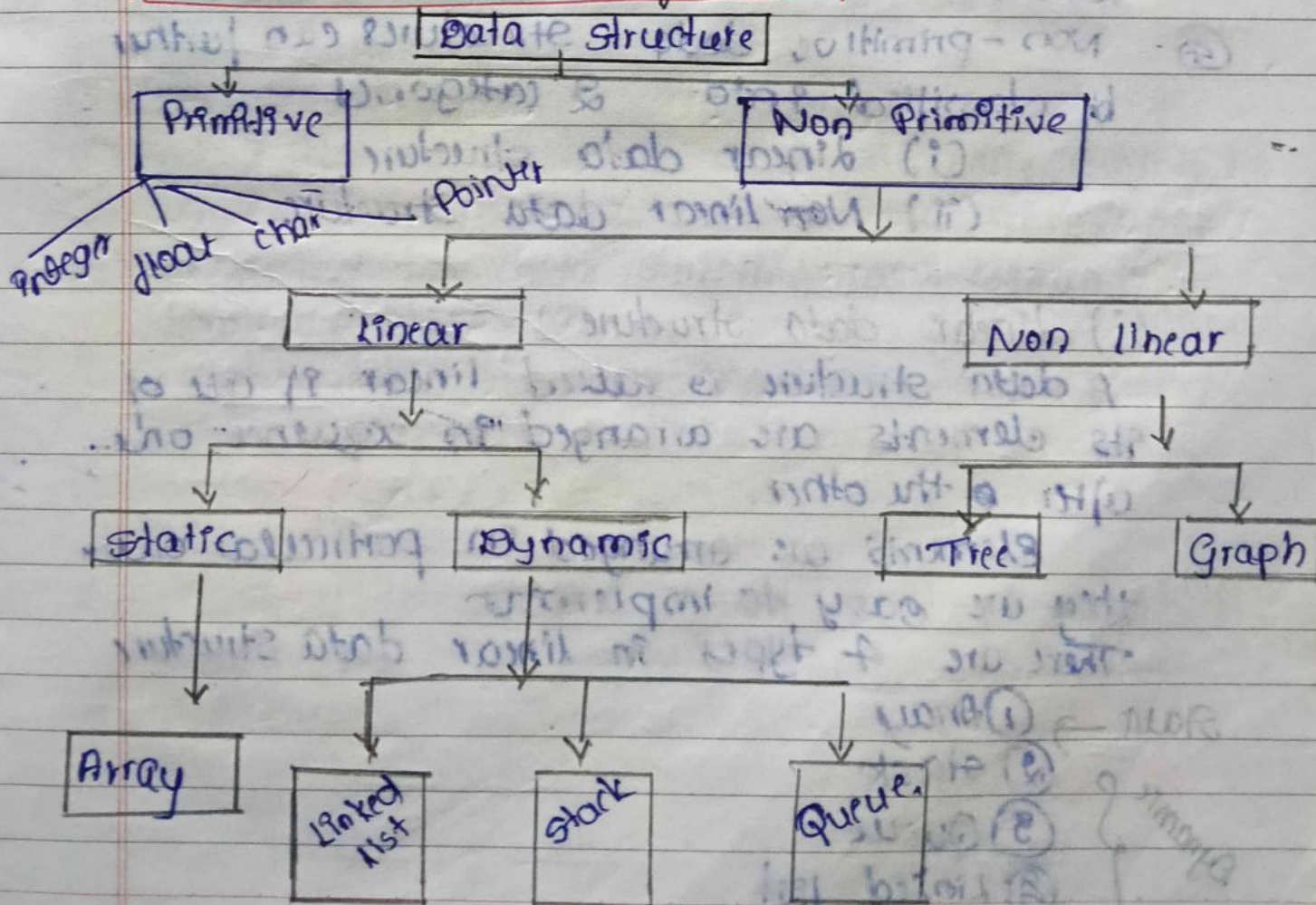
Definition of Data structure

* Data structure can be defined as the group of data elements which provides an efficient way of storing and organising data in the computer so that it can be used efficiently.

* Data structure is representation of the logical relationship existing between individual elements of data.

* It is a way of arranging data on a computer so that it can be accessed and updated efficiently.

Data Structure Classification



Primitive data Structure

- ① Primitive data structures are the fundamental data types which are supported by a programming language.
 - ② Primitive data structures are primitive data types.
 - ③ Some basic data types are **int**, **float**, **char** and **pointer** are hold a single value.
- Non Primitive data structures
- ① The data type derived from primary data types are known as non-primitive data types.
 - ② Non-primitive data structures are those data structures which are created using primitive data structure.
 - ③ These data types are used to store group of values.
 - ④ Non-primitive data structures can further be classified into 2 categories.
 - (i) Linear data structure
 - (ii) Non linear data structure
- (i) Linear data structure
- A data structure is called linear if all of its elements are arranged in sequence one after the other.
- Elements are arranged in particular order. they are easy to implement. there are 4 types in linear data structure.
- Stack →
- ① Array
 - ② stack
 - ③ Queue
 - ④ linked list

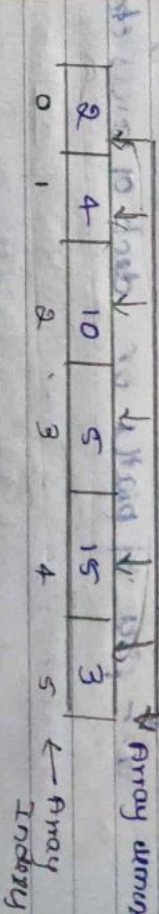
Types of linear data structure

- ① Arrays - An array is a collection of similar type of data items and each data item is called an element of the array.
- An array is defined as a set of finite number of homogeneous elements on same data items.

* Array is a collection of homogeneous elements with same name and same data type.

* The array can be one dimensional, two dimensional, multi dimensional.

array



② Linked list

- ① linked list is a linear data structure which is used to maintain a list in the memory.
- ② In linked list data structure data elements are connected through a series of nodes and each node contains the data items and address to next node.

Eg:-



③ start

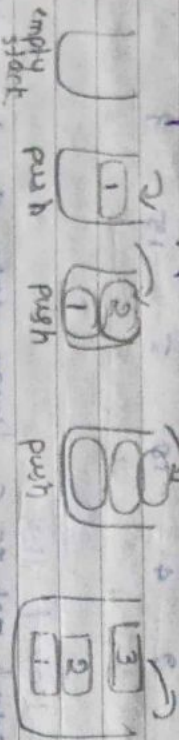
① starts as a linear list in which insertion & deletions are allowed only at one end, called top

② A stack is a linear data structure that follows the LIFO [last in first out] principle.

③ It is a simple data structure that allows adding and removing elements in a particular order.

④ operation is used to add the element is push().

⑤ pop() is used to delete the element



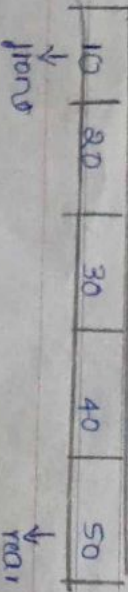
④ Queue

Queue is linear list in which elements can be inserted by only at one end, called rear and deleted only at the other end, called front.

Queue is opened at both ends, therefore it is known as first-in-first-out (FIFO) methodology for storing the data items.

When first elements store in the queue will be remove first.

Ex: think of a queue at the railway station ticket counter.



(ii) Non linear data structure

* The data elements are not arranged in sequential structure

* Instead they are arranged in a hierarchical manner where one element will be connected to one or more elements

There are 3 types in non-linear data structure

① Graph

② Tree

① Trees → graph without simple circuit. The first node in a tree is a non-linear data structure, the root at first.

* Tree is also collection of vertices and edge. However in tree data structure, there can only be one edge b/w two vertices.

* A tree is a collection of nodes connected by directed or (undirected) edges.

* popular tree based data structure

① Binary tree → each parent consists of child.

② Binary search tree

③ The node which predecessor of any node

child - the node which is descendant of any node

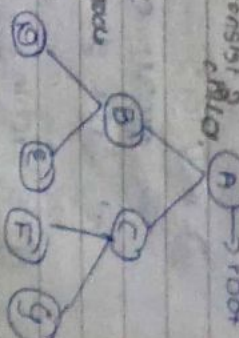
siblings - node belongs to same parent no child - leaf

③ Graphs → non empty set of edges & vertices

* A graph is a non-linear data structure consisting of vertices and edges.

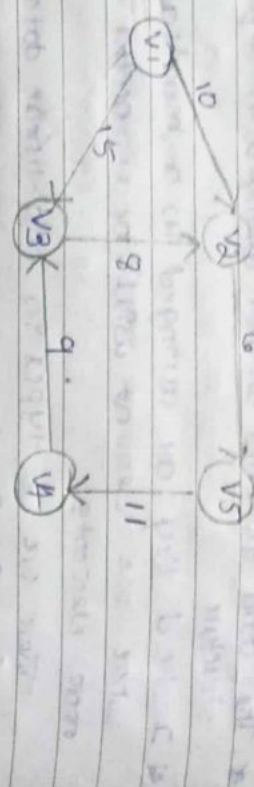
* In graph data structure, each node called vertices and each vertices connected to other vertices through edges.

* Graph can be defined as the pictorial representation of the set of elements, (represented by vertices) connected by links known as edges.



Types of graph → Directed graph, Undirected graph, Weighted graph, Unweighted graph

The graph is denoted by $G(V, E)$



OPERATIONS ON DATA STRUCTURES

① Traversing:

This operation is used to visit every single element in the data structure.

② Insertion:

Insertion can be defined as the process of adding the elements to the data structure at any location.

③ Deletion:

The process of removing the elements from the data structure is called deletion.

④ Searching:

The process of finding the location of an element within the data structure is called searching.

There are 2 algorithms to perform searching.

- ① Linear Search
- ② Binary Search

⑤ Sorting: The process of arranging the data structure in a specific order is known as sorting.

There are many easy algorithms that can be used to perform sorting. ex: Insertion sort, selection sort, bubble sort.

⑥ Merging:

- * It is used to combine the data items of two sorted list in a single list. In the sorted form.
- * When two list A and list B of size N and N respectively of similar type of element, clubbed.

STRUCTURE

* A structure is a composite data type that defines a grouped list of variables that are to be placed under one name in a block of memory.

Ex :- Structure is a group of elements with different data types that falls under a same name.

* It is convenient tool for handling a grouping of related data items of different types.

* Syntax :-

struct structure-name
{
 datatype member1;
 datatype member 2;
}

datatype member 1;
datatype member 2;

Ans: A structure is a collection of different datatype grouped together under a single name.

Ex-1:

```
struct employee
{
    int id;
```

```
    char name[20];
    float salary;
```

```
};
```

Ex 2: struct student

```
{
```

```
    int rollno;
```

```
    char name[50];
```

```
    float percentage;
```

```
};
```

Declaring a structure variable

struct structure-name

```
{
```

```
    datatype member1-name;
```

```
    datatype member2-name;
```

```
    datatype member n-name;
```

```
};
```

Example

```
struct student
```

```
{
```

```
    int roll no;
```

```
    char name[20];
```

```
    float percentage;
```

```
};
```

Accessing the element of a structure.

* The member of structure can be accessed using dot operator.

* It is also called as memory access operator

* Syntax: Variable - name . member - name;

Example:-

```
struct student
```

```
{
```

```
    int rollno;
```

```
    char name[50];
```

```
    float percentage;
```

```
};
```

```
int main()
```

```
{
```

```
    struct student s1;
```

```
    s1.roll no = 123456789;
```

```
    s1.name = "Ravi Kumar";
```

```
    s1.percentage = 85.5;
```

```
    return 0;
```

```
}
```

```
}
```

```
}
```

① Write a Program to store n students information using structure.

```
#include <stdio.h>
struct students {
    char name[60];
    int rollNum;
    char address[75];
    float marks;
};
int main() {
    int n;
    printf("Enter number of students: ");
    scanf("%d", &n);
    struct students s[n];
    for (int i = 0; i < n; i++) {
        printf("Enter roll number: ");
        scanf("%d", &s[i].rollNum);
        printf("Enter name: ");
        scanf("%s", s[i].name);
        printf("Enter address: ");
        scanf("%s", s[i].address);
        printf("Enter marks: ");
        scanf("%f", &s[i].marks);
    }
    printf("Student Information\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%s\t%s\t%f\n", s[i].rollNum, s[i].name, s[i].address, s[i].marks);
    }
}
```

```
int main() {
    int n;
    printf("Enter number of students: ");
    scanf("%d", &n);
    struct students s[n];
    for (int i = 0; i < n; i++) {
        printf("Enter roll number: ");
        scanf("%d", &s[i].rollNum);
        printf("Enter name: ");
        scanf("%s", s[i].name);
        printf("Enter address: ");
        scanf("%s", s[i].address);
        printf("Enter marks: ");
        scanf("%f", &s[i].marks);
    }
    printf("Student Information\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%s\t%s\t%f\n", s[i].rollNum, s[i].name, s[i].address, s[i].marks);
    }
}
```

```
printf("Addition");
puts(s[i].address);
printf("Marks: %f", s[i].marks);
printf("\n");
return 0;
```

Initializing a structure member → direct
There are two ways to do this: using initialization

① Using dot(.) operator - (in many lines)

Syntax: - var.name.member = value;

Ex: - s1.age = 18;

s1.gname = 'M';

② all members assigned in one statement

Syntax - struct structure-name {

datatype member1;

datatype member2;

datatype member3;

datatype member4;

}; Variable name = { values for all members;

ex: - struct students / array s[n];

int rollNo;

char name[60];

float percentage;

{ s1 = { 7, "Verma", 85 };

s2 = { 8, "Sharma", 78 };

s3 = { 9, "Kumar", 92 };

Structure Variable (in function)

local variable

① Program to illustrate inside a structure
#include <stdio.h>
#include <string.h>
struct student
{
char name[35];
int age; // 10
char branch[10];
char gender; // 1
};
// SI ← variable (inside along with structure)
int main() {
// ← main function
}

int age; // 10
char branch[10];
char gender; // 1
// SI ← variable (inside along with structure)
int main() {
// ← main function
}

method 2

② Program to illustrate variable outside a structure (in different file)

#include <stdio.h>
#include <string.h>
struct student
{
char name[35];
int age;
char branch[10];
char gender;
};
int main() {
// ← variable declaration outside structure (after main function)
SI-age = 18;
strcpy(SI-name, "Vivek");
strcpy(SI-branch, "BBA");
SI-gender = 'M';
printf("age of the student: %d\n", SI-age);
printf("name of the student: %s\n", SI-name);
printf("branch of the student: %s\n", SI-branch);
printf("gender of the student: %c\n", SI-gender);
return 0;
}

Array as a Structure.

Method 1: Array of structure initialization

* We can also declare an array of structure variables.
* In which each element of the array will represent a structure variable

Example: struct - employee emp[5];

struct - student st[10];

Method 2: declaring (Array of structure) in main function (with in main() function)

// Array of structure initialization in c

#include <stdio.h>

struct employee

{
int age;
char name[50];
float salary;
};

int main()
{
struct employee emp[4];
emp[0] = {25, "Veraj", 25000};
emp[1] = {24, "Vikra", 35000};
emp[2] = {22, "Vinay", 28000};
emp[3] = {27, "Vani", 35000};
}

};

emp[0] = {25, "Veraj", 25000};
emp[1] = {24, "Vikra", 35000};
emp[2] = {22, "Vinay", 28000};
emp[3] = {27, "Vani", 35000};

main function()

Nested Structure

Method 1: one structure can be declared inside another structure

A nested structure in c is a structure within a structure. one structure can be declared inside another structure in the same way structure members are declared inside a structure

Syntax: struct name - 1

{
member 1;
member 2;
member n;
}

struct name - 2

{
member 1;
member 2;
member n;
}

member - 1;
member 2;
member n;
};

Ex:- struct student

{
char name[30];
int age;
struct address;
char city[50];
int pincode;
float gaddr;
};

Ex 2: struct students

{
char name[30];
int age;
struct date;
};

struct date;
};

Union

Union is a data type in C programming that allows different data types to be stored in the same memory locations.

Union provides an efficient way of using the memory location, as only one of its members can be accessed at a time.

Syntax:

```
union union-name
{
    datatype member1;
    datatype member2;
    datatype member n;
```

Program:

Union program Ex: To illustrate union

```
#include <stdio.h>
#include <union.h>
```

```
union number
```

```
{
```

```
    int n1;
```

```
    float n2;
```

```
};
```

```
int main()
```

```
{
```

```
    union number x;
```

```
    printf("Enter the value of n1: ");
```

```
    scanf("%d", &x.n1);
```

```
    printf("Value of n1 = %d", x.n1);
```

```
printf("\nEnter the value of n2: ");
scanf("%f", &x.n2);
printf("Value of n2 = %f/n", x.n2);
```

```
    return 0;
```

```
}
```

enter a value of n1 = 10

value of n1 = 10

enter a value of n2 = 20

value of n2 = 20.

Pointer

defn * Pointer is a variable which contains the address of another variable.

* It may pointer to variable of any data type like integer, char etc.

* The two key symbol concerned with pointers are ' & ' - address of and * ' - pointed at by.

* These two symbols are called as address operator & indirection operator.

Key points to remember about pointer

1. Normal variable stores the value where as pointer variable store the address of the variable.
2. The content of the C pointer always be a whole number that is address.
3. Always C pointer is initialized to null that is `int *p = null;`
4. The value of null pointer is 0.
5. ' & ' symbol is used to get the address of the variable.
6. * symbol used to get the value of variable that the pointer is pointing to.

7. If a pointer in C is assigned to null, it means it is pointing to nothing

Program to illustrate Pointers

```
#include <stdio.h>
int main()
{
    int *ptr, q;
    q = 50;
    ptr = &q;
    printf("%d", *ptr);
}
```

O/P - 50

declaring and initializing pointers

Syntax of pointer

datatype *pointer-name;

int *ptr;

Lab Program - 01

① Write a program to create, initialize and access a pointer variable.

```
#include <stdio.h>
int main()
{
    int a, *p;
```

// declare an integer variable 'a' & a pointer variable 'p'.

p = &a; // the pointer 'p' holds the address of the variable 'a'.

```
printf("%d\n", *p);
printf("%d\n", &a);
printf("%d\n", p);
printf("%d\n", &p);
return 0;
```

O/P
1000000006
21F81C
21F81C
21F81C

// Print the value stored at the address that 'p' is pointing to (which is the value of 'a').

// Print the memory address of variable 'a'.

// Print the memory address of pointer 'p'.

Output of the program: address of the pointer 'p' is 21F81C, address of 'a' is 1000000006, value of 'a' is 10.

Address of 'a' is 1000000006, value of 'a' is 10.

1000000006
21F81C
21F81C
21F81C

C pointers

A pointer can be used to store the memory address of other variables, functions or even other pointers.

Syntax of pointers — `datatype *ptr;`

How to use pointers?

The use of pointers in C can be divided into 3 steps

- ① Pointer declaration
- ② Pointer initialization
- ③ Pointer Dereferencing

① Pointer declaration

In pointer declaration, to declare a pointer we use the (*) dereference (indirection) operator before its name.

Ex — `int *ptr;`

② Pointer initialization

Pointer initialization is the process where we assign some initial value to the pointer variable. We generally use (&) address of operator to get the memory address of a variable & then store it in the pointer variable.

Ex `int var = 10;`
`int *ptr;`
`ptr = &var;`

③ Pointer Dereferencing

Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. We use the same (*) dereferencing operator that we used in the pointer declaration.

ptr

`0x711a057dd4`

`0x71198b49e8` — Address of pointer variable ptr

var — value of variable var (*ptr)

`0x711a057dd4` — Address of variable var (stored at ptr)

③

C program to calculate length of the string using #include <stdio.h>

#include <stdio.h>

int main()

{

char s[10], *p;

int i;

printf("Enter a string: ");

scanf("%s", s);

while (*p != '\0')

{

p++;

p++;

printf("length of string is %d", i);

return 0;

}

O/P.

Enter a string

'Neha'

length of

string is 4

Lab program - 03

③ Write a program to swap two numbers using pointers.

#include <stdio.h>
#include <conio.h>

int main()

{
int a, b, c, *p, *q;

printf("Enter the numbers :");

scanf("%d %d", &a, &b);

p = &a;
q = &b;

printf("before swapping a=%d b=%d", *p, *q);

*p = *q;

*q = *p;

*p = *q;

printf("after swapping a=%d b=%d", *p, *q);

return 0;

}

O/P
Enter the numbers: 10 20
before swapping a=10 b=20
after swapping a=20 b=10

Memory Management Function

① block of memory in each block having bytes of memory. int a[5] [20 bytes]

② Static memory allocation - Fixed size, compile time.

③ Dynamic

not fixed run time allocation size. ex: stack, queue, linked list

char name[10] → [n][1] [1] [1] [1] [1] [1] [1] [1] [1] [1]

malloc() → continuous user allocate memory

calloc()

realloc()

free()

Memory Management Function - This is function

also allocate block of memory & the size of the block is remember by byte

Two types

① Static memory allocation

② Dynamic memory allocation

① Static memory allocation → memory size fixed

* This method allocate the memory during compile time is called static memory allocation

* The allocate memory space cannot be expanded to accommodate no data or cannot be

reduced to accommodate less data

* Static memory allocation is performed in an array int a[10]; which allocate 80 bytes to variable a.

Dynamic memory allocation

② Dynamic memory allocation

- * It is the process of allocating memory during execution time.
- * It is allocation techniques used pre-defined function to allocate & release memory for data during execution time.
- * Dynamic memory allocation preferred in linked list.

Dynamic memory allocation function

1. malloc()
2. calloc()
3. free()
4. realloc()

Lab Program - 04

Program: Write a program in C to print all permutations of a given string using pointer.

```
#include <stdio.h>
#include <string.h>
void swap(char*ch1, char*ch2)
```

```
{
    char tmp; tmp=*ch1; *ch1=*ch2; *ch2=tmp;
}
```

```
void permute(char*a, int k, int r)
{
    if (k==r) printf("%s\n", a);
    else
```

```
for (i=k; i<=r; i++)
    swap(a+k, a+i);
    permute(a, k+1, r);
}
```

```
{
    swap(a+1, a+1);
```

```
permute(a, 1+1, r);
swap(a+1, a+1);
```

```
int main()
{
    char str[10];
    printf("Enter string\n");
    gets(str);
    int n = strlen(str);
    printf("The permutations of string are:\n");
    permute(str, 0, n-1);
    printf("\n");
    return 0;
}
```

```
printf("The permutations of string are:\n");
permute(str, 0, n-1);
printf("\n");
return 0;
```

```
a b c
```

```
o/p
entri string
```

```
abc
```

The permutation of string are

abc

acb

bac

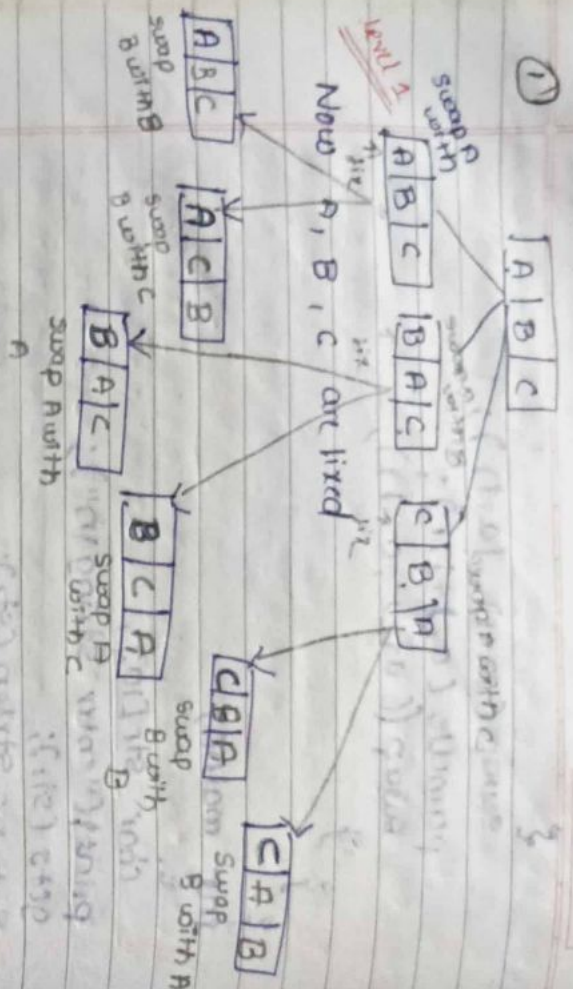
bca

cba

cab

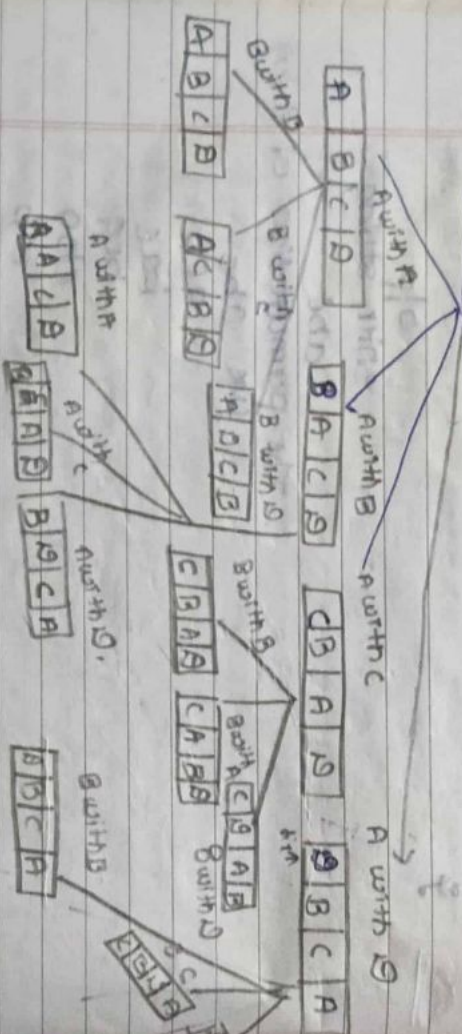
Find the permutation

①



Total - we get 6 permutations

②



12

1. malloc() memory allocation

- * This function allocate block of memory
- * The size of the number of bytes specified in the parameter
- * This function will not initialize during execution as it leaves garbage value and it returns new pointer

Syntax → `ptr = (datatype *) malloc (size);`

eg: `Mptr = (int *) malloc (100 * size (rns));`

- * In the above example, the statement allocate 800 bytes of memory because the int size is 8 bytes and the variable mptr points to the address of the first byte of the memory.
- * On success allocation the function returns the address of first byte of allocation memory
- * on failure condition is called over flow in memory such case function returns null.

Program

`malloc()` example

#include <stdio.h>

#include <stdlib.h>

int n, i, *ptr;

printf("Enter the total no of values: ");

scanf("%d", &n);

ptr = (int *) malloc (n * size of (int));

printf("Enter the values: ");

for (i = 0; i < n; i++)

scanf("%d", &ptr[i]);

```

printf("\n entered value are");
for(i=0; i<n; i++)
{
    printf("%d", *(ptr+i));
}
free(ptr);
}

```

2. calloc()

- * This is also known as contiguous allocation
- * This function is used to allocate multiple block of memory
- * As in malloc() will not initialize any memory bit but calloc() it allocate the memory along with initializing the bits zero.
- * Syntax - Ptr = (data type *) calloc (n, size);
- Example :- ptr = (int *) calloc (35, size of int);
- * In this above example statement allocates contiguous memory space for about 35 elements of data type "int"
- * On such allocation the function require returns the address of first byte of allocation memory
- * On failure condition is called Overflow memory in such case function written null.
- * The total number of byte allocated is $n * \text{size}$, all bytes will be initialized to zero.

```

2. calloc()
Program #include <stdio.h>
#include <stdlib.h>
int main()
{
    int n, i, *ptr;
    printf("Enter the total no of values :");
    scanf("%d", &n);
    ptr = (int *) calloc (n, sizeof(int));
    printf("\n Enter the values :");
    for (i=0; i<n; i++)
    {
        scanf("%d", (ptr+i));
    }
    printf("\n the entered value are");
    for (i=0; i<n; i++)
    {
        printf("%d", (ptr+i));
    }
    printf("\n the entered values are");
    for (i=0; i<n; i++)
    {
        printf("%d", *(ptr+i));
    }
    free(ptr);
}

```

3.

Realloc()

- Before using this function memory should have been allocated using malloc() or calloc() function.
- Realloc() changes the size of the block by extending or deleting the memory at the end of the block.

- If the existing memory can be extended pointer value will not change.

- Syntax - `realloc (pointer variable, n);`

ex | `ptr = (datatype *) malloc (ptr, n * size);`
`char *ptr;`

`Rptr = (char *) malloc (200);`
`Rptr = realloc (Rptr, 400);`

Program

`#include <stdio.h>`

`#include <stdlib.h>`

`int main()`

`{`
`int n, i, *ptr, *ptr1;`

`printf("Enter the total no of values:");`

`scanf("%d", &n);`

`ptr = (int *) malloc (n * size of (int));`

`printf("Enter the values:");`

`for (i=0; i<n; i++)`

`{`

`scanf("%d", (ptr+i));`

`}`

`printf("\n Enter the update size of n:");`

`scanf("%d", &n);`

`ptr1 = (int *) realloc (ptr, n * size of (int));`

`printf("\n previous address: %d, new address: %d",`

`ptr, ptr1);`

`printf("\n Enter the value are:");`

`for (i=0; i<n; i++)`

`{`

`printf("%d", *(ptr+i));`

`}`

`free (ptr);`

`}`

free()

- In dynamic memory allocation, malloc() and calloc() function only allocate memory but doesn't free the memory on their own.

- So this is done using free() method explicitly to release the memory that is not in use avoid memory leakage.

Syntax - `free (pointer-variable);`

Example : `free (ptr);`

`#include <stdio.h>`

`#include <stdlib.h>`

`int main()`

`{`

`int *p, *p1;`

`int i, j;`

`i=5;`

`printf("Enter the number of elements to allocate in memory: %d\n", i);`

`p = (int *) malloc (i * size of (int));`

`p1 = (int *) malloc (j * size of (int));`

`if (p = NULL || p1 = NULL)`

`{`

`printf("Memory is not allocated\n");`

`exit (0);`

`}`

`printf("Memory has been allocated using malloc\n");`

free (p);

printf("malloc memory has been successfully freed or released\n");

printf("memory has been allocated using malloc\n");

free (p);

printf("cause memory has been successfully freed or released\n");

return 0;

};

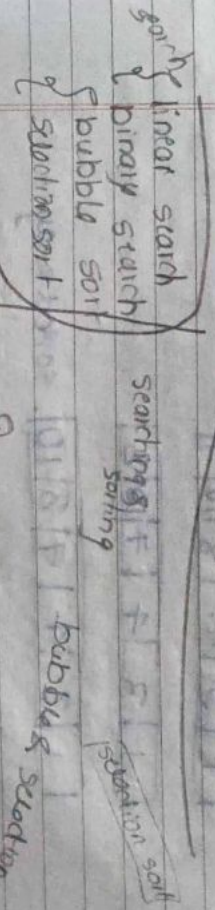
Difference between static and dynamic

static	dynamic
1. memory allocated during compile time	1. Memory allocated during runtime
2. size of the memory to be allocated is fixed	2. As on when memory is required memory can be allocated, if not required can be deallocated (size not fixed)
3. Execution time is faster since memory is already allocated	3. Execution time is slower since memory has to be allocated during runtime
4. Memory is allocated in stack area	4. Memory is allocated only heap area
5. example - array	5. example - linked list

Difference b/w malloc & calloc

malloc()	calloc()
1. Malloc function allocate a single block of memory of size bytes	1. calloc function allocate multiple block of memory each block with same size
2. Syntax: ptr = (datatype*) malloc(size);	2. Syntax: ptr = (datatype*) calloc(n, size);
3. Allocated space will not be initialized memory is not initialized	3. Each byte of allocated space is initialized to zero memory
4. Since no initialization takes place time efficiency is higher than calloc	4. calloc is more expensive because of zero filling

error



Sorting - bubble
Sorting - selection
Sorting - insertion
Sorting - bubble
Sorting - selection
Sorting - insertion
Sorting - bubble
Sorting - selection
Sorting - insertion

Data Structures - on d/fn

NAME: _____
DATE: / /

Meaning
↓
well organised data
↓
Storing of collected data used to store & organise data

Data Structure

Types

Primitive

- ① Int
- ② float
- ③ char

directly work with machine/memory
single data elements.

Non primitive

string, array, not directly see data-type, next convert

Linear

Non-linear

Sequence

nonsequence

4 kinds

- ① Arrays
- ② Stacks
- ③ Queue
- ④ Linked lists

allocated during (compile) Static

(run) Dynamic

connect the number of series in nodes.

last in first out
first in first out
placing books
web browser
Hyper link

Scattered sequence
collection, stored different
store in anywhere but not in sequential

Trees

two nodes connected through edge.

graphs

non empty sets of edge & vertices.

collect, organise, work, remove add. all
Process are completely discuss in Data structure

write a Program to store n students information using structure array and, stack

Unit 3

chapt 1 -

Stacks

10M - 11M
5M - push & pop

* A data structure that works on the principle of last-In-First-Out (LIFO) / First-In-Last-Out (FILO)

* i.e. there the item which entered last will be the item which will be removed first

* Memory allocation / representation

ex: consider a glass of water
(i) whenever we define a stack, the stack is having the value 'NULL'.

(ii) In a stack we can perform any operation from only one end i.e. from top

(iii) So we define a variable TOP , with initial value $TOP = NULL$

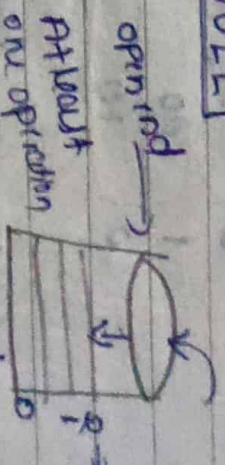
Basic operations

① Push (Insertion)

(i) If 1st element is being inserted, then make $TOP = 0$, & then insert

(ii) If elements are already present then increment $TOP + 1$ (TOP by one) to make position for new item

(iii) If stack is full, check $TOP \geq MAXSIZE$, then stop inserting (overflow).



① PUSH (Insertion)

Algorithm: Push element at the top of the stack.

- ① if $TOP \geq MAXSIZE$, then print "Stack full".
- ② else

$TOP = TOP + 1$

$stack[TOP] = item$

- ③ end if
- ④ stop.

② POP (deletion)

Algorithm: Pop element from the top of the stack.

- ① Before deleting check if stack is empty.

if $TOP = NULL$ then

stop deleting (underflow)

- ② otherwise, decrementing TOP by 1.

4	30	40	50
3	20	30	40
2	10	20	30
1	0	10	20
0			

Algo for POP

- ① if $TOP = NULL$, then print "Stack empty".

- ② else $item = stack[TOP]$ - store for future if you want.
- $TOP = TOP - 1$

- ③ end if

- ④ stop.

③ PEEK

returning/displaying topmost item.

Algorithm

if $TOP = NULL$ then print "Stack empty"

- ② else

return $stack[TOP]$

- ③ end if
- ④ stop.

④ is Empty()

Checks if stack is empty.

Algorithm

if $TOP = NULL$, then

return 1

- ② else return 0

- ③ end if
- ④ stop.

⑤ is Full()

Checks if stack is full.

Algorithm

if $TOP = NULL$ then

return 1

- ② else

return 0

Representation of stack

There are two ways to implement stacks.

- ① static (fixed) memory allocation
- ② dynamic memory allocation

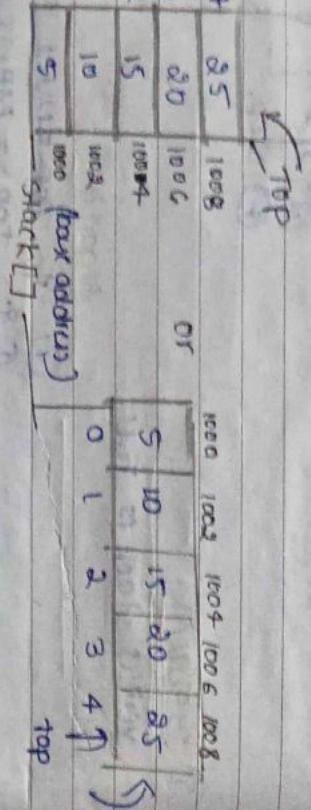
① Static (fixed) memory allocation

- * the memory size is fixed before using
- * that is the size of any data structure can not be altered.

* here to store data we use continuous memory allocation

* So to implement this feature we use single dimensional array.

Ex: Consider $int\ stack[5] = \{5, 10, 15, 20, 25\}$



② Dynamic memory allocation

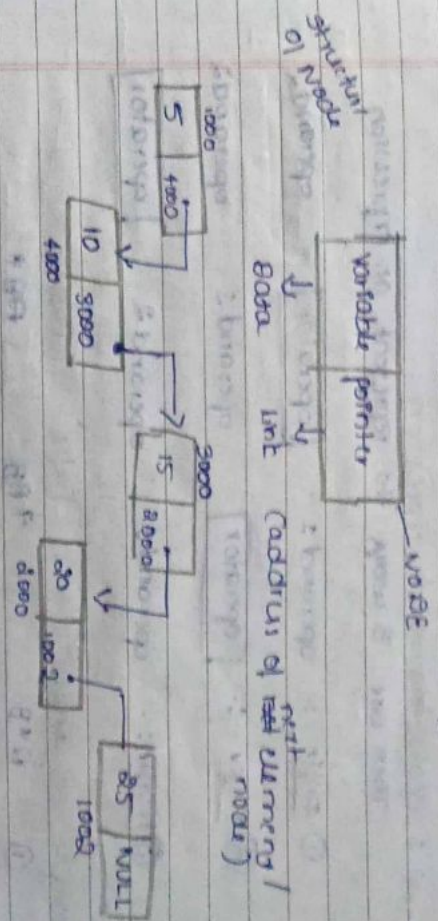
- * To solve the problem of memory wastage or requirement of additional memory during execution can be solved using dynamic memory allocation.

* Here the memory is allocated when asked or required.

* because of this concept memory is not allocated in continuous form that is it is used discontinuous memory allocation.

* to implement this feature we use singly linked list.

* Structure of singly linked list (SLL)



Appl:

$$1 - 10 + 50 = 40 - 20 + 10 = 30$$

Application of Stacks

1. Evaluating and expression
2. Function call management
3. Reversing a string
4. Decimal to binary conversion
5. Backtracking

1. Evaluating and expression

There are 3 ways to represent an expression

- ① Infix :- operand 1 operator operand 2
- ② Prefix :- operator operand 1 operand 2
- ③ Postfix :- operand 1 operand 2 operator

Example.

	Infix	Prefix	Postfix
①	$A * B$	$*AB$	$AB*$
②	$(A+B)/C$	$/+ABC$	$AB+C/$
③	$(A*B) + (C-D)$	$+*AB-CD$	$AB*CD-+$

- Precedence
- ① $()$
 - ② $\$, \wedge, \%$
 - ③ $+$
 - ④ $-$

1. Converting Infix to postfix

① $a+b*c+d$ to postfix

Step 1: $a+b*c+d$ → $a+b*c+d$

Step 2: $a+b*c+d$ → $a+b*c+d$

Step 3: $a+b*c+d$ → $a+b*c+d$

Step 4: $a+b*c+d$ → $a+b*c+d$

Step 5: $a+b*c+d$ → $a+b*c+d$

Step 6: $a+b*c+d$ → $a+b*c+d$

Step 7: $a+b*c+d$ → $a+b*c+d$

Step 8: $a+b*c+d$ → $a+b*c+d$

Step 9: $a+b*c+d$ → $a+b*c+d$

Step 10: $a+b*c+d$ → $a+b*c+d$

Note: ① When an operand is encountered, add it to the stack and push it to the resulting expression

For this example, consider "postfix" as the final expression

- ② When an operator is encountered, push it into the stack according to the operators precedence. That is push higher precedence operators above lower precedence operators

Example: $a+b*c+d$

Step 1: $a+b*c+d$ → $a+b*c+d$

Step 2: $a+b*c+d$ → $a+b*c+d$

Step 3: $a+b*c+d$ → $a+b*c+d$

Step 4: $a+b*c+d$ → $a+b*c+d$

Step 5: $a+b*c+d$ → $a+b*c+d$

Step 6: $a+b*c+d$ → $a+b*c+d$

Step 7: $a+b*c+d$ → $a+b*c+d$

Step 8: $a+b*c+d$ → $a+b*c+d$

Step 9: $a+b*c+d$ → $a+b*c+d$

Step 10: $a+b*c+d$ → $a+b*c+d$

③ $a+b*c+d$ → $a+b*c+d$

Step 1: $a+b*c+d$ → $a+b*c+d$

Step 2: $a+b*c+d$ → $a+b*c+d$

Step 3: $a+b*c+d$ → $a+b*c+d$

Step 4: $a+b*c+d$ → $a+b*c+d$

Step 5: $a+b*c+d$ → $a+b*c+d$

Step 6: $a+b*c+d$ → $a+b*c+d$

Step 7: $a+b*c+d$ → $a+b*c+d$

Step 8: $a+b*c+d$ → $a+b*c+d$

Step 9: $a+b*c+d$ → $a+b*c+d$

Step 10: $a+b*c+d$ → $a+b*c+d$

④ $a+b*c+d$ → $a+b*c+d$

Step 1: $a+b*c+d$ → $a+b*c+d$

Step 2: $a+b*c+d$ → $a+b*c+d$

Step 3: $a+b*c+d$ → $a+b*c+d$

Step 4: $a+b*c+d$ → $a+b*c+d$

Step 5: $a+b*c+d$ → $a+b*c+d$

Step 6: $a+b*c+d$ → $a+b*c+d$

Step 7: $a+b*c+d$ → $a+b*c+d$

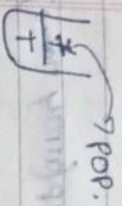
Step 8: $a+b*c+d$ → $a+b*c+d$

Step 9: $a+b*c+d$ → $a+b*c+d$

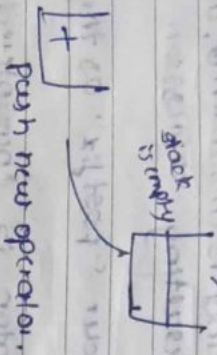
Step 10: $a+b*c+d$ → $a+b*c+d$

v) $i = 5$, $expl[i] = '+'$

(new push this operator into stack.)



postfix = abc * +



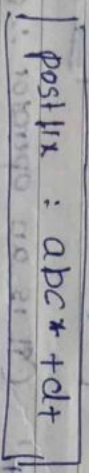
but there are already operators present in the stack, $\therefore$ before inserting the new operator check the precedence of present operators, if new operator precedence is lower than the present operators then pop (remove) the present operators in the LIFO manner and add it to final expression.

(vii) $i = 6$, $expl[i] = 'd'$

$\Rightarrow$ postfix = abc * + d

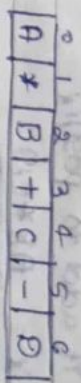
(viii) $i = 7$, $expl[i] =$ no elements present

$\therefore$ empty stack and pop all operators and add to final expression.



postfix = abc * + d +

(x) $(A * B) + (C - D)$



'i' is counter as counter variable

(i) $i = 0$, $expl[i] = 'A'$ (since A is operand add to final expression)

(ii) $i = 1$, $expl[i] = '*'$ (since it is operator push it in stack)

(iii) $i = 2$, $expl[i] = 'B'$ (since B is operand add to final expression)

(iv) $i = 3$, $expl[i] = '+'$ (since it is operator push it in stack)



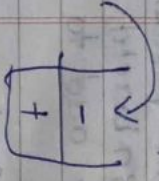
already * is there so precedence is higher it is also in bracket * have lower precedence.



higher precedence (more upper) remove * and add to postfix because it has higher precedence than +

(v) $i = 4$, $'C' \rightarrow$ postfix = AB * C

(vi) $i = 5$, $'-'$



'-', having higher precedence than + but '-' is in bracket so it has high value than + of '+' is higher because it is inside brackets

IMP

Infix to Prefix

$$① A + (B + C - (D / E * F) * G) * H$$

$$= A + (BC + - (D / E * F) * G) * H$$

$$= A + (*BC - / * E F) * G * H$$

$$= A + (*BC - * / * E F G) * H$$

$$= A + - * BC * / * E F G * H$$

$$A + * - * BC * / * E F G H$$

$$= [A + * - * BC * / * E F G H]$$

$$② P + (R * S) / (T - U) [(A + (B * C) / (D - E))]$$

$$P + (* R S / - T U) [(A + (* B C / - D E))]$$

$$[(A + / * B C - D E)]$$

$$[+ A / * B C - D E]$$

$$- + A / * B C D E$$

$$③ A * (B + C + D * E) + F$$

$$④ A \uparrow B * C - D + E / F / (G + H)$$

$$⑤ (A + B) * C + (D - E) / F + G$$

$$① A \uparrow B * C - D + E / F / (G + H)$$

$$A \uparrow B * C - D + E / F / + G H$$

$$\uparrow A B + C - D + / E F / + G H$$

$$\uparrow A B * C - D + / E F + G H$$

$$* \uparrow A B C - D + / / E F + G H$$

$$* \uparrow A B C - + D / / E F + G H$$

~~DEF~~

$$- * \uparrow A B C + D / / E F + G H$$

3 Postfix to Infix

① $AB + C * DE - F + G +$

$(A + B) C * (D - E) F + G +$

$(A + B) * C (D - E) / F + G +$

$(A + B) * C + (D - E) / F G +$

$(A + B) * C + (D - E) / F + G$

② $AB(C/D) * + F(AE) + -$

$AB(C/D) * + F(AE) + -$

$AB * (C/D) + F + (AE) -$

$A + B * (C/D) F + (AE) -$

$A + B * C/D - F + (AE)$

③ Infix to postfix

$P + (Q * R) / (S - T)$

$P + (Q * R) / (S - T)$

$P + Q * R - / +$

$PQR * ST - / +$

2. postfix to infix

$PQR * ST - / +$

$PQR * ST - / +$

$PQR * ST - / +$

$P(Q * R) / (S - T) +$

$P(Q * R) / (S - T) +$

$P + (Q * R) / (S - T)$

④ $ABC * DE F / G * - H * +$

$A(B * C) D(E * F) / G * - H * +$

$A(B * C) D / (E * F) G * - H * +$

$A(B * C) D / (E * F) * G - H * +$

$A(B * C) D / E * F$

$A(B * C) - D / (E * F) * G * H * +$

$A(B * C) - D / (E * F) * G * H * +$

$A(B * C) - D / (E * F) * G * H * +$

$A + (B * C - (D / (E * F) * G) * H)$

4 Prefix to Infix

① $+ P / * Q R - S T$

$+ P / (Q * R) (S - T)$

$+ P (Q * R) / (S - T)$

$= [P + ((Q * R) / (S - T))]$

② $+ * A + * B C * DE F$

$+ * A + (B * C) (D * E) F$

$+ * A ((B * C) + (D * E)) F$

$+ A * ((B * C) + (D * E)) F$

$A * (B * C) + (D * E) + F$

③ $- * A B C + D / E F + G H$

$- * A B C + D / (E F) + G H$

$- * A B C + D / (E F) + G H$

$- * A B C + D / (E F) + G H$

$- * A B C + D / (E F) + G H$

2017-2018

Positive to Total

10/11/2019

$$[x/c]$$

(c)(2)

(3) $\text{C}(\text{C}^*)$ is

last step

postfix-infix

1000000

$$\frac{(7-2)(1+5)}{16} = 94$$

11

10

204

ति २०११ अ० ३३

[illegible]

1. 246746

Preface

1975-1976 - position

31. *Spencer spread up*

$$(a+b)(r+r)$$
$$C(\theta + \theta_0) =$$

an index

1933

$$100 + 80 + 100 + 20 + 80$$
$$-7 - 3x + 4 = 8$$

2. 11-12-13-14-15-16-17-18-19-20-21-22-23-24-25-26-27-28-29-30-31-32-33-34-35-36-37-38-39-40-41-42-43-44-45-46-47-48-49-50-51-52-53-54-55-56-57-58-59-60-61-62-63-64-65-66-67-68-69-70-71-72-73-74-75-76-77-78-79-80-81-82-83-84-85-86-87-88-89-90-91-92-93-94-95-96-97-98-99-100-101-102-103-104-105-106-107-108-109-110-111-112-113-114-115-116-117-118-119-120-121-122-123-124-125-126-127-128-129-130-131-132-133-134-135-136-137-138-139-140-141-142-143-144-145-146-147-148-149-150-151-152-153-154-155-156-157-158-159-160-161-162-163-164-165-166-167-168-169-170-171-172-173-174-175-176-177-178-179-180-181-182-183-184-185-186-187-188-189-190-191-192-193-194-195-196-197-198-199-200-201-202-203-204-205-206-207-208-209-210-211-212-213-214-215-216-217-218-219-220-221-222-223-224-225-226-227-228-229-230-231-232-233-234-235-236-237-238-239-240-241-242-243-244-245-246-247-248-249-250-251-252-253-254-255-256-257-258-259-260-261-262-263-264-265-266-267-268-269-270-271-272-273-274-275-276-277-278-279-280-281-282-283-284-285-286-287-288-289-290-291-292-293-294-295-296-297-298-299-300-301-302-303-304-305-306-307-308-309-310-311-312-313-314-315-316-317-318-319-320-321-322-323-324-325-326-327-328-329-330-331-332-333-334-335-336-337-338-339-340-341-342-343-344-345-346-347-348-349-350-351-352-353-354-355-356-357-358-359-360-361-362-363-364-365-366-367-368-369-370-371-372-373-374-375-376-377-378-379-380-381-382-383-384-385-386-387-388-389-390-391-392-393-394-395-396-397-398-399-400-401-402-403-404-405-406-407-408-409-410-411-412-413-414-415-416-417-418-419-420-421-422-423-424-425-426-427-428-429-430-431-432-433-434-435-436-437-438-439-440-441-442-443-444-445-446-447-448-449-450-451-452-453-454-455-456-457-458-459-460-461-462-463-464-465-466-467-468-469-470-471-472-473-474-475-476-477-478-479-480-481-482-483-484-485-486-487-488-489-490-491-492-493-494-495-496-497-498-499-500-501-502-503-504-505-506-507-508-509-510-511-512-513-514-515-516-517-518-519-520-521-522-523-524-525-526-527-528-529-530-531-532-533-534-535-536-537-538-539-540-541-542-543-544-545-546-547-548-549-550-551-552-553-554-555-556-557-558-559-560-561-562-563-564-565-566-567-568-569-570-571-572-573-574-575-576-577-578-579-580-581-582-583-584-585-586-587-588-589-590-591-592-593-594-595-596-597-598-599-600-601-602-603-604-605-606-607-608-609-610-611-612-613-614-615-616-617-618-619-620-621-622-623-624-625-626-627-628-629-630-631-632-633-634-635-636-637-638-639-640-641-642-643-644-645-646-647-648-649-650-651-652-653-654-655-656-657-658-659-660-661-662-663-664-665-666-667-668-669-670-671-672-673-674-675-676-677-678-679-680-681-682-683-684-685-686-687-688-689-690-691-692-693-694-695-696-697-698-699-700-701-702-703-704-705-706-707-708-709-710-711-712-713-714-715-716-717-718-719-720-721-722-723-724-725-726-727-728-729-730-731-732-733-734-735-736-737-738-739-740-741-742-743-744-745-746-747-748-749-750-751-752-753-754-755-756-757-758-759-760-761-762-763-764-765-766-767-768-769-770-771-772-773-774-775-776-777-778-779-780-781-782-783-784-785-786-787-788-789-790-791-792-793-794-795-796-797-798-799-800-801-802-803-804-805-806-807-808-809-810-811-812-813-814-815-816-817-818-819-820-821-822-823-824-825-826-827-828-829-830-831-832-833-834-835-836-837-838-839-840-841-842-843-844-845-846-847-848-849-850-851-852-853-854-855-856-857-858-859-860-861-862-863-864-865-866-867-868-869-870-871-872-873-874-875-876-877-878-879-880-881-882-883-884-885-886-887-888-889-890-891-892-893-894-895-896-897-898-899-900-901-902-903-904-905-906-907-908-909-910-911-912-913-914-915-916-917-918-919-920-921-922-923-924-925-926-927-928-929-930-931-932-933-934-935-936-937-938-939-940-941-942-943-944-945-946-947-948-949-950-951-952-953-954-955-956-957-958-959-960-961-962-963-964-965-966-967-968-969-970-971-972-973-974-975-976-977-978-979-980-981-982-983-984-985-986-987-988-989-990-991-992-993-994-995-996-997-998-999-1000-1001-1002-1003-1004-1005-1006-1007-1008-1009-1010-1011-1012-1013-1014-1015-1016-1017-1018-1019-1020-1021-1022-1023-1024-1025-1026-1027-1028-1029-1030-1031-1032-1033-1034-1035-1036-1037-1038-1039-1040-1041-1042-1043-1044

...

3

11 ~~left~~ to positive

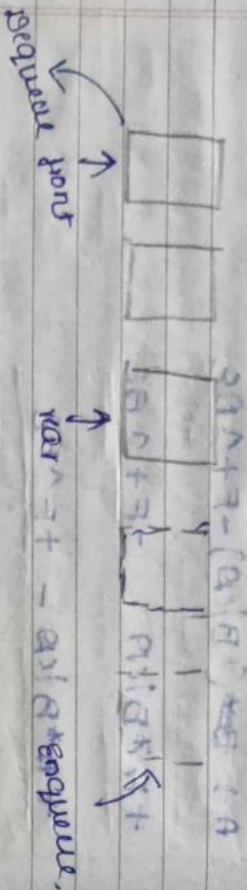
Queues

1 Principle - FIFO

This is a special kind of data structure where items are inserted at one end called 'rear' (rear) and deleted at another end called 'front' - (3rd + 3rd) (4th + 3rd) + 3rd

This data structure works on 'FIFO' principle that is 'First In First Out'

- ① Enqueue - Insertion (word used for insertion)
- ② Dequeue - Deletion $(+3 - (2nd * 2) + 1)$

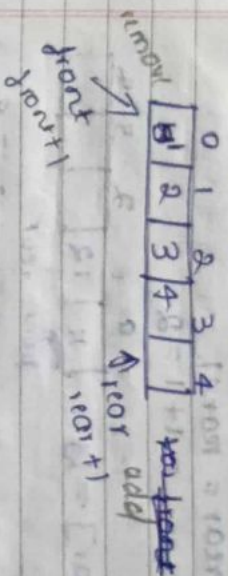


2 Representation of Queue using Array
 Consider an array
 ind queue [5] = {11, 12, 13, 14, 15}

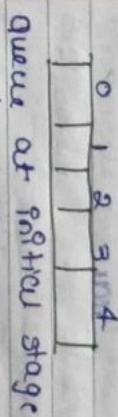
Initial stage
 front = 0
 rear = -1



Inserting first element towards back

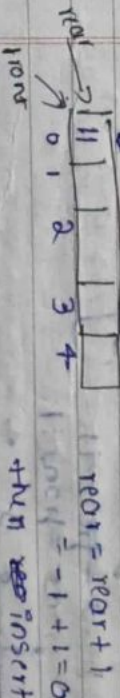


Initial queue [5] = {11, 12, 13, 14, 15}

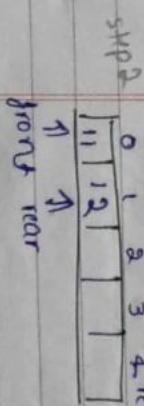


Position time (enqueue)

Whenever new item is being inserted, increment rear by 1, to make position for the new element



next insertion, rear + 1



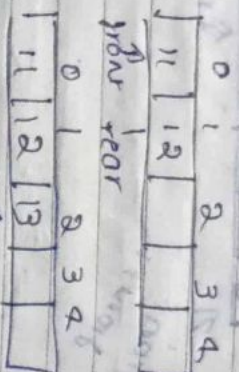
queue [rear] = 12

11) 3rd element to be inserted

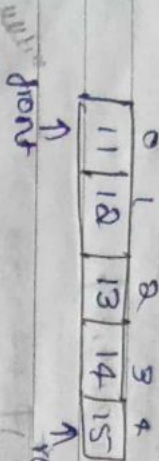
$$\text{rear} = \text{rear} + 1$$

$$\text{rear} = 1 + 1 = 2$$

queue [rear] = 13



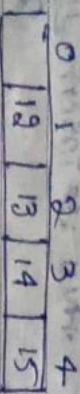
likewise when we insert all elements in queue.



deletion time
(dequeue)

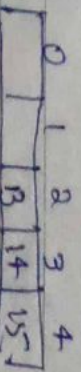
⇒ whenever we delete item from the queue, we remove it from front and that is we increment front by 1

{ item = queue [front] } optional part to delete item is



to delete next item

$$\text{front} = \text{front} + 1$$



2 exceptions related to Queue

① Overflow

$$\text{rear} = \text{maxsize} - 1$$

$$\text{rear} \geq \text{maxsize}$$

② Underflow when front = 0, & rear = -1

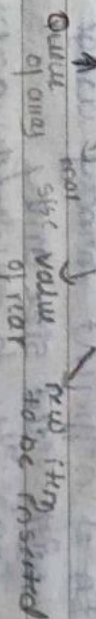
$$\text{front} > \text{rear}$$

Basic Operations

(with algorithms)

→ ① Enqueue (Insertion)

enqueue (Q, n, r, e) ← Parameters required



if (rear) > max size

(rear = max size - 1)

print "Queue is full"

else

$$r = r + 1$$

$$\text{queue}[r] = e \leftarrow \text{new item}$$

end if

stop

→ 2. ② Regressive Algorithm

(j, n, q)

if ($j > r$) then
print "Queue empty"

else

if ($r == 0$) then
print "Queue empty" (optional)

if ($j == r$) then
if ($j == 0$, $r = -1$)

else

$j = j + 1$

end else
stop

Problems be face while working with simple

① ~~Not~~ Inefficient memory usage

② Inability to efficiently access elements

other than the front, leading to time consuming operations

③ Lack of priority handling

Types of Queues

→ ① Circular Queue

Principle :- The last element is connected to the first, allowing efficient memory utilization when the queue has fixed size.

ex - ① a circular buffer used in data streaming

→ ② Priority Queue

Principle: the elements are assigned priority, and the element with the highest priority is dequeued first.

ex - in hospitals, emergency rooms

→ ③ Double ended Queue (Deque)

Principle - Insertion and deletion from both ends (front & rear)

ex :- A deck of cards where you can add or remove cards from either the top or bottom

I Circular Queues:

Principle - the last element is connected to the first allowing efficient memory utilisation when the queue has fixed size

It - A circular buffer used in data streaming

It solve the major limitation of the normal queue that is wastage of memory

Circular Queue works by the priority of circular queue increment that is when we try to increment the round (rear - front)

we reach the end of the queue we starts from the beginning of the queue.

for position

rear = (rear + 1) % maxsize

for deletion

front = (front + 1) % maxsize

Algorithms

Initialization

- Set $\text{front} = \text{rear} = -1$, Queue is empty
- Define maxsize

Enqueue Algorithm

```

1) if  $(\text{rear} + 1) \% \text{maxsize} == \text{front})$  then
    print "Queue full"
else
    if  $(\text{front} = \text{rear} = -1)$  then
         $\text{front} = 0$  // for empty queue
         $\text{rear} = (\text{rear} + 1) \% \text{maxsize}$ 
        Queue  $\text{Enqueue} = \text{item}$ 
    end else if
    stop.
    
```

Dequeue Algorithm

```

1) if  $(\text{front} == -1)$  then
    print "Queue empty"
else
    if  $\text{front} = \text{Queue}[\text{front}]$ 
        // using pointer
        if  $(\text{front} = \text{rear})$  then
             $\text{front} = -1$ ,  $\text{rear} = -1$ 
        else
             $\text{front} = (\text{front} + 1) \% \text{maxsize}$ 
    stop.
    
```

Enqueue

if $(\text{next_rear}) = \text{front}$

print "Queue is full"

else

if (empty)

$\text{front} = 0$

if $(\text{front} == -1)$

Dequeue

print "Queue is empty"

else

$\text{front} = \text{Queue}[\text{front}]$

Ans.

To show working of circular queue.

Q [6]

$\text{front} = \text{rear} = -1$

Enqueue (10)

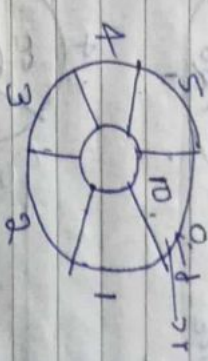
$\text{rear} = 0 \% 6 = 0$

$\text{front} = 0$

$\text{rear} = 0$

Q [rear] = 10

① enqueue (10)



(ii)

Enqueue (80) Similarly after inserting last element enqueue (30)

$\text{rear} = (0 + 1) \% 6 = 1 \% 6$

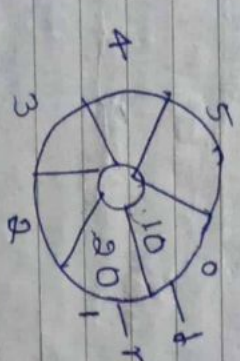
$\text{rear} = 1$

Q [rear] = 80

$\text{rear} = (1 + 1) \% 6 = 2 \% 6$

$\text{rear} = 2$

Q [rear] = 50



(iv) enqueue(60)
 $q[(5+1) \% 6]$
 $6 \% 6 = 0$

4th

Print "queue is full"

(v) Dequeue()

item = Queue(Front)

= Queue[0] = 10

front = (front + 1) % maxsize

= (0 + 1) % 5

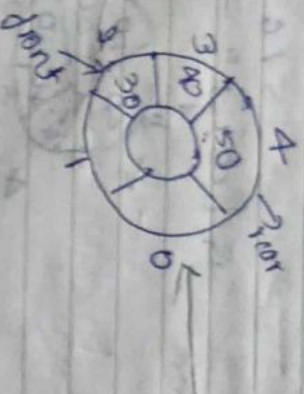
front = 1 % 5 = 1

front = 1

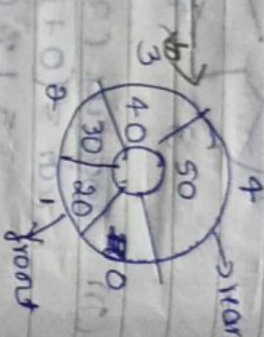
(vi) Dequeue()

item = 20

front = 2



(vii) After two times dequeue

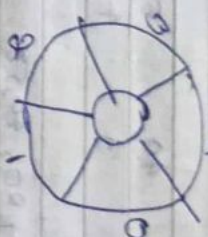


(viii) Dequeue()

item = 50

Print "can see that front = rear" i.e. single element present

front = 1



II. Priority Queues

- * Every element is associated with a priority.
- * The elements are served in the order of highest to lowest priority
- * If multiple elements have the same priority, they are served as per their original order.

Representation

* Ordered queue. (For faster deletion)

* Consider a queue where elements are in the form 10, 20, 30, 40, 50

* They are stored as per their priority during enqueue operation

* So the highest priority elements are served quickly from the front side.

Priority	3	1	3	2	2	1
current	30	10	30	20	40	50
Index	0	1	2	3	4	

Dequeue operation will be: 50, 40, 40, 40, 50

② Unordered Queue (for faster insertion)

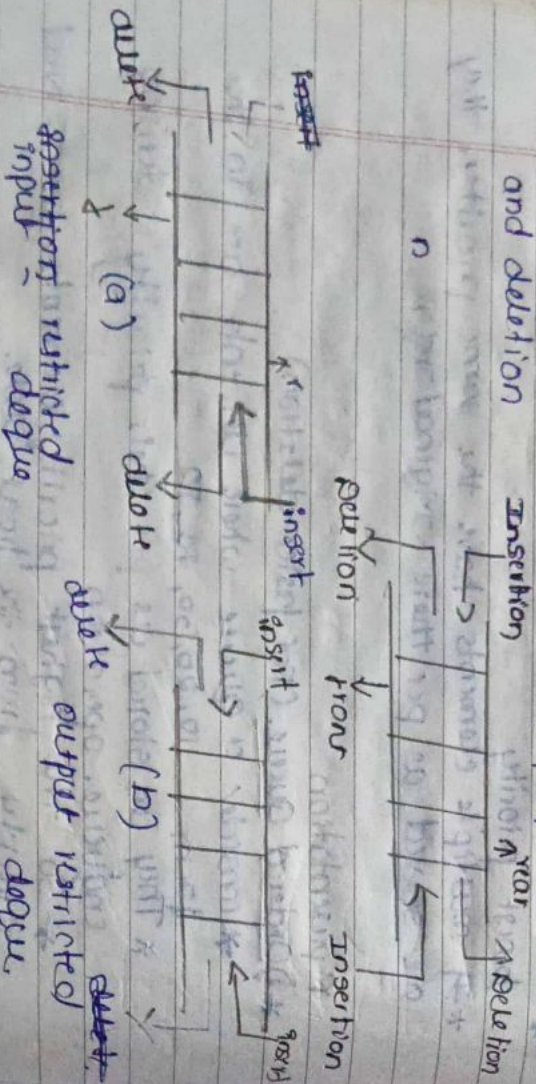
The queue operation serves the highest priority elements by searching the entire queue

Priority	15	18	3	18	11
element	10	80	30	40	50
index	0	1	2	3	4

Representation will be 10, 30, 80, 40, 50

~~Queue~~ DEQUE - Double Ended Queue

* From both the ends we can perform insertion and deletion



Types of DEQUE

(a) Input restricted deque

- * Insertion is allowed only from one end, from rear end
- * Deletion can be done from both ends

(b) Output restricted deque

- * Deletion is allowed from one end, from front
- * Insertion can happen from both ends

Inherits the properties of both Queues & Stacks

Representation of Deque using circular queues

* We will be using circular array for implementing deque that is if the array is full, we again start from the beginning.

* The first element is considered as the next of the last element

* We can perform 4 operations: (1) Insertion at front (2) Deletion at rear

(3) Insertion at rear (4) Deletion at front

(5) Deletion at rear (6) Insertion at front

Initial setup: front = rear = -1

① Insertion at front (enqueue front)

if (front == -1)

if (front == 0 & rear == n-1) then

print "Queue Full"

else if (front == -1 & rear == -1) then

front = rear = 0

deque[front] = item

else if (j == 0) then

j = n-1
 BQ[j] = item

else

BQ[j] = item



② Insertion at rear end.

if (j == 0) && r == n-1 || (j+1) then
 print "Queue is full"

else if (j == -1 && r == -1) then

BQ[r] = item

else if (r == n-1) then

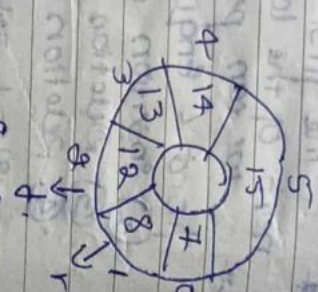
r = 0

BQ[r] = item

else

r++

BQ[r] = item



Delete front:-

① Condition: queue should not be empty

if (j == -1 && r == -1), then

print "Queue empty"

else

item = BQ[f]

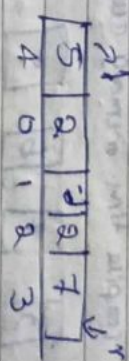
if (j == n-1), then

j = 0

else if (j == 1) then, add j+1

j = j + 1

stop.



Delete rear

① condition: queue should not be empty

if (j == -1 && r == -1) then print "Queue empty"

else

if (r == 0) then

r = n-1

else if (j == r) then

j = r-1

stop.

stop.

stack operation
 conversion - array to post
 initial top = 1

enqueue operation
 under queue

dequeue operation
 under queue

Queue

NAME		TOTAL MARKS
CLASS	SUBJECT	
ROLL No.	DATE	

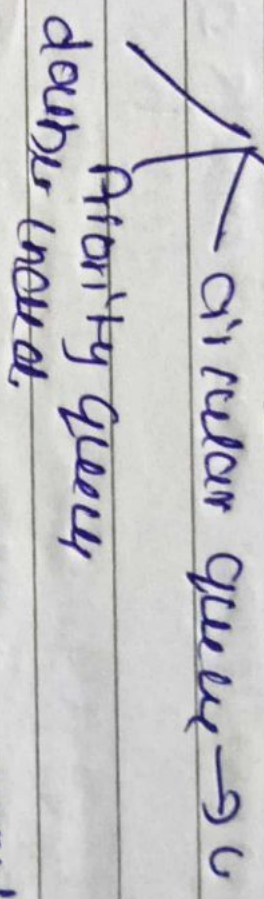


① defn queue

② Representation of queue.

③ operation on queue - insert, delete, display

④ Types of queue



circular queue - defn, program, working
priority queue - " "
double linked queue - defn - types